

1) Two classes, user and player, were too similar, had the same fields, had mostly the same methods and covered the same things.



aryagith opened last week · edited by sleimon

Edits

...

- **Code Smell ID:** 1
- **User Story:** User Profile and Chips/Virtual Currency Management
- **Description:**

The `User` and `Player` classes are redundant and contain overlapping functionality and data. Both classes store similar data (e.g., chip count, wins, losses), which leads to redundancy and unnecessary complexity. The duplication of this data violates the **DRY (Don't Repeat Yourself) principle** and creates unnecessary coupling between the two classes. This design also leads to potential issues with data synchronization and consistency between the two classes. It also makes the code in the game class much less readable and has contributed to some of the bugs in the game class.
- **Refactoring Suggestion:**

Merge the `User` and `Player` classes into a single class. This will remove redundancy and improve code maintainability by consolidating the related data into one place. The class should manage the user-specific data (e.g., chips, wins, losses) while keeping logic related to gameplay (e.g., betting, win/loss outcomes) in the same class.
- **Severity:** 🚨 High

Create sub-issue

🔖

aryagith added `code smell` last week

👤

sleimon assigned [sleimon](#) and [aryagith](#) last week

🔗

sleimon added 3 commits that reference this issue 5 days ago

Laid the foundation for the player class to absorb the user class and...

498df12

Laid the foundation for the player class to absorb the user class and...

494bb12

Added a new feature, lifetime wins/losses and session wins/losses

6b132e0

📄

sleimon mentioned this 9 hours ago

[Added a new feature, lifetime wins/losses and session wins/losses #60](#)



sleimon 9 hours ago

Owner

...

Added a new feature that adds a different identity to the player class

Now the player class keeps track of session wins/losses

And the user class keeps track of lifetime wins and losses

Which makes each class non-redundant

👍

🏆

sleimon closed this as `completed` 9 hours ago

To solve this problem, we split the functionalities of both classes such that they had their own identities and nothing was overlapping.

2) Our game class was over 700 lines long and handled both business logic and GUI functionalities

## # Code Smell: Large Class #31

Closed

aryagith opened last week · edited by sleimon

Code Smell ID: 3

- **User Story:** Different Cards (Deck, Hand, and Game Functionality) and Betting
- **Description:**  
The `Game.java` class is too large and handles too many responsibilities, including **GUI, game logic, and statistics management**. This violates the **Single Responsibility Principle (SRP)** and makes the code hard to maintain.
- **Refactoring Suggestion:**  
Break `Game.java` into separate classes:
  - `GameGUI` for GUI-related logic
  - `GameLogic` for game rules and mechanics
  - `GameStats` for statistics tracking
- **Severity:** 🚨 High

Create sub-issue

sleimon assigned sleimon and aryagith last week

aryagith added code smell last week

sleimon mentioned this 2 days ago

Code Smell: Long Methods: UI, Authentication, and Game Logic in Main Class #41

sleimon removed their assignment 1 hour ago

sleimon 1 hour ago

Owner

Closed by Arya in his most recent pushes to his branch (and eventually the main branch).  
He split the game class into two classes (`gameLogic` and `gameGUI`) which solved the large class issue

sleimon closed this as completed 1 hour ago

We solved this by splitting the game class into two classes, `gameLogic` and `gameGUI`, one handles the business logic and the other handles the GUI.

3) A method in the game class was around 300 lines and covered multiple functions such as creating buttons, assigning JLabels and setting up event listeners

## # Code Smell: Large Method (setupGUI in Game.java) #32

 Closed

 **aryagith** opened last week · edited by sleimon Edits ▾ ...

**Code Smell ID: 4**

- **User Story:** Different Cards (Deck, Hand, and Game Functionality) and Chips/Virtual Currency Management
- **Description:**  
The `setupGUI()` method in `Game.java` is too long and handles too many responsibilities, including button creation, event handling, and game statistics display.
- **Refactoring Suggestion:**  
Split `setupGUI()` into smaller methods, each with a single responsibility, such as:
  - `createButtons()`
  - `addEventListeners()`
  - `displayGameStats()`
- **Severity:**  Low

Create sub-issue ▾ 

  **aryagith** added `code smell` last week

  **sleimon** assigned [sleimon](#) and [aryagith](#) last week

  **sleimon** added 2 commits that reference this issue 2 days ago

Fixed some ticket [#32](#), code smell ID [#4](#) cc56bb2

Merge pull request [#58](#) from [sleimon/sleiITR3](#) Verified bea1de5

 **sleimon** 2 days ago Owner ...

Split the `setupGUI()` method into three methods.

- `setupGUI()` handles the size of the `JPanel`, displaying the card pngs and the creation and placement of all the `JLabels`
- `createButtons()` handles the creation of the many buttons as well as the size and placement of them
- Lastly, `actionListeners` handle all the event listeners on all the buttons



We solved this problem by splitting the `setupGUI` method into three

`setupGUI` handles the size of the `JPanel`, the card pngs and the creation and placement of all the `JLabels`

`createButtons()` handles the creation of the many buttons as well as the size and placement of them

Lastly, `actionListeners` handle all the event listeners on all the buttons